GREEN LIFE - HAVE KNOWLEDGE. PAY ATTENTION.

Green Life Session Outputs Complete

This document consolidates the major product, design, architecture, API, and implementation artifacts created in this session.

1. Startup Package

Executive Summary

- Green Life is a mobile app that helps users identify the environmental impact of products and materials around them.
- Users scan a barcode, take a photo, or search by name to view a source-backed impact summary, including carbon footprint, material composition, recyclability, disposal guidance, and confidence level.
- The problem is fragmented environmental information across EPA resources, EPD databases, manufacturer disclosures, and other sustainability datasets.
- The solution is a fast, mobile-first transparency layer that normalizes environmental data and clearly distinguishes exact records from estimates.
- The target market includes environmentally conscious consumers, homeowners/renters, and shoppers comparing products.
- Long-term monetization could include affiliate recommendations, premium consumer features, a B2B sustainability lookup API, and enterprise procurement/reporting tools.

MVP Feature Spec

Feature 1: Barcode Scan

Purpose:

Allow fast identification of packaged products.

Requirements:

- User can open camera and scan barcode.
- App sends barcode to backend.
- Backend attempts exact product match.
- If found, app returns product profile with environmental data.
- If not found, app offers search fallback.

Success Metric:

- high scan-to-result completion rate on supported products

Feature 2: Text Search

Purpose:

Allow lookup when barcode is missing or scan fails.

Requirements:

- User can search by product name, brand, material, or category.
- Search returns best-match results ranked by relevance.
- User can select a result and open detail view.

Success Metric:

- high rate of successful product/category discovery from search

Feature 3: Photo Capture / Image Input

Purpose:

Support future object identification and OCR-assisted matching.

MVP Scope:

- capture/upload image
- optionally run OCR for visible labels/text
- use image as a fallback input, not primary matching method

Success Metric:

- assists search resolution when barcode is unavailable

Feature 4: Impact Summary Screen

Purpose:

Present environmental information clearly and quickly.

Requirements:

- Show product/item name
- Show match type: exact / probable / estimate
- Show key metrics:
- carbon footprint
- material composition
- recyclability
- recycled content
- disposal guidance
- hazard flags where available
- Show source attribution for each metric
- Show freshness/date where available

Success Metric:

- users understand the result without needing to open technical details

Feature 5: Confidence Labeling

Purpose:

Prevent overstating precision.

Requirements:

- Every result must display one of:
- Exact Product Match
- Probable Product Match
- Category Estimate
- Material Estimate
- Confidence label must appear near top of result screen.
- Metrics derived from estimates should be visibly marked.

Success Metric:

- low user confusion and low trust erosion from uncertain data

Feature 6: Saved Items / History

Purpose:

Improve retention and repeat use.

Requirements:

- User can save an item
- User can view prior scans/searches
- User can remove saved entries

Success Metric:

- repeat engagement and retained value after first use

Feature 7: Admin Review Tool

Purpose:

Support internal quality control.

Requirements:

- Admin can inspect product-source mappings
- Admin can review questionable matches
- Admin can correct category/product associations
- Admin can view data provenance and confidence metadata

Success Metric:

- reduced error rate in user-facing results

Technical Architecture

Architecture Overview

Use a mobile client backed by a normalization and matching platform.

Recommended stack:

- Mobile: React Native with Expo
- Backend API: FastAPI or NestJS
- Database: PostgreSQL
- Search: PostgreSQL full-text initially
- Storage: S3-compatible object storage
- Auth: Clerk, Firebase Auth, or Auth0
- Admin UI: simple React web panel

Core Components

Mobile App

- onboarding/auth
- barcode scanner
- camera/photo upload
- search UI
- result screen
- saved history

API Layer

- authentication
- product lookup endpoints
- search endpoints
- saved item endpoints
- source/metric retrieval

Matching Service

- barcode exact match
- search ranking
- OCR-assisted product matching
- fallback category/material estimation

Environmental Scoring Service

- retrieves normalized impact metrics
- resolves conflicts between sources
- attaches confidence score
- formats results for API clients

Data Ingestion / ETL

- imports EPA and other datasets
- parses EPD and structured disclosures
- maps incoming records to canonical schema
- stores provenance and freshness data

Admin Tooling

- review records
- inspect source lineage
- override mappings
- QA uncertain results

Lookup Flow

1. User scans barcode or searches text.
2. API sends request to matching service.
3. Matching service resolves product or fallback category/material.
4. Scoring service retrieves environmental metrics.
5. API returns normalized response with item identity, metrics, source attribution, confidence label, and timestamps/freshness.

2. Product Requirements Document

- Full dedicated PRD remains in PRD.md / PRD.docx.
- It includes 21 sections covering overview, problem statement, vision, goals, non-goals, target users, user stories, core product experience, MVP scope, functional requirements, data requirements, data sources, trust principle, success metrics, UX requirements, technical requirements, risks, assumptions, open questions, release plan, and launch criteria.

3. Screen Map

Primary App Structure

- Home
- Search
- Saved
- Profile

Primary entry actions from Home:

- Scan Barcode
- Take Photo
- Search Product

Secondary flows:

- Login / Sign Up
- Permissions
- Result Details
- Source Details
- Error / No Match
- Save Confirmation

Screen Inventory

- Onboarding
- Permissions
- Auth
- Home
- Barcode Scanner
- Search
- Photo Capture
- Candidate Results
- Result Details
- Metric Detail
- Source Details
- No Match
- Saved
- Profile
- Admin Web Panel

Core Navigation Flow

- Onboarding -> Permissions -> Auth or Guest -> Home
- Home -> Barcode Scanner -> Result Details or No Match
- Home -> Search -> Result Details or No Match
- Home -> Photo Capture -> Candidate Results -> Result Details or No Match
- Result Details -> Metric Detail
- Result Details -> Source Detail
- Result Details -> Saved

4. API Specification

Principles

- All results must include confidence metadata.
- All environmental metrics must include provenance where available.
- The API must distinguish exact product data from category/material estimates.
- Missing metrics should return partial success, not hard failure.

Base path:

`/api/v1`

Core Domain Models

ProductSummary

```
{
  "id": "prod_123",
  "name": "Reusable Water Bottle",
  "brand": "Example Brand",
  "image_url": "https://...",
  "match_type": "exact_product",
  "confidence_score": 0.96,
  "category": {
    "id": "cat_10",
    "name": "Drinkware"
  }
}
```

ImpactMetric

```
{
  "metric_type": "carbon_footprint",
  "label": "Carbon Footprint",
  "value": 2.4,
  "unit": "kg_co2e",
  "scope": "product",
  "estimate_type": "exact",
  "confidence_score": 0.91,
  "methodology": "EPD reported cradle-to-gate value",
  "source": {
    "id": "src_55",
    "name": "EPD Database",
    "source_type": "epd",
    "reference_url": "https://...",
    "dataset_version": "2026-01",
```

```
      "published_at": "2026-01-12"
    }
  }
```

ResultDetail

```
  {
    "id": "prod_123",
    "name": "Reusable Water Bottle",
    "brand": "Example Brand",
    "description": "Stainless steel insulated bottle",
    "image_url": "https://...",
    "match_type": "exact_product",
    "confidence_score": 0.96,
    "metrics": [],
    "materials": [],
    "disposal_guidance": [],
    "hazard_flags": [],
    "sources": [],
    "last_updated_at": "2026-03-05T10:00:00Z"
  }
```

Public/User Endpoints

- POST /lookup/barcode
- GET /search
- POST /lookup/image
- GET /items/{id}
- GET /sources/{id}

User Account Endpoints

- GET /me/history
- POST /me/saved-items
- GET /me/saved-items
- DELETE /me/saved-items/{itemId}
- GET /me/profile

Admin Endpoints

- GET /admin/products
- GET /admin/products/{id}
- GET /admin/source-records
- GET /admin/match-review
- POST /admin/mappings/override

5. Database Schema

This is a practical MVP schema for PostgreSQL. It is designed around product matching, environmental metrics with provenance, and separation between exact data and estimates.

Key Tables

- users
- brands
- categories
- materials
- products
- product_identifiers
- product_materials
- sources
- source_records
- product_source_links
- impact_metrics
- disposal_guidance
- hazard_flags
- saved_items
- lookup_history
- mapping_overrides
- review_queue

Service Rules

6. Prefer exact product metric.
7. Fall back to verified product mapping.
8. Fall back to category metric.
9. Fall back to material metric.
10. Return best result with confidence and estimate label.

6. Frontend Scaffold

Stack

- Expo
- TypeScript
- React Navigation
- TanStack Query
- Zustand

- Expo Camera
- Expo Image Picker
- react-native-safe-area-context
- react-native-svg

Folder Structure

- src/api
- src/navigation
- src/screens
- src/components/ui
- src/components/item
- src/components/metrics
- src/theme
- src/types
- src/utils

Core Screens

- HomeScreen
- SearchScreen
- BarcodeScannerScreen
- PhotoCaptureScreen
- CandidateResultsScreen
- ItemDetailScreen
- MetricDetailScreen
- SourceDetailScreen
- NoMatchScreen
- SavedScreen
- ProfileScreen
- OnboardingScreen
- PermissionsScreen
- AuthScreen

Shared Components

- Screen
- PrimaryButton
- SecondaryButton
- SearchBar
- ResultCard
- MatchBadge
- ConfidenceBadge

- MetricRow
- EmptyState
- ErrorState
- LoadingState
- StatCard

7. Figma-Ready Screen Spec

Pages

11. 00 Cover
12. 01 Foundations
13. 02 Wireframes
14. 03 UI Kit
15. 04 Screens
16. 05 Prototype
17. 06 Notes

Required Screens

- Onboarding
- Permissions
- Home
- Search Empty
- Search Results
- Barcode Scanner
- Photo Capture
- Candidate Results
- Item Detail Exact
- Item Detail Estimated
- Metric Detail
- Source Detail
- No Match
- Saved
- Profile

Prototype Flows

- Home -> Barcode Scanner -> Item Detail -> Source Detail
- Home -> Search -> Item Detail -> Metric Detail
- Home -> Photo Capture -> Candidate Results -> Item Detail
- Item Detail -> Saved

8. Engineering Backlog

Frontend

- build shared UI primitives
- wire search and item detail
- implement barcode and image flows
- add loading/empty/error states
- polish auth and saved flows

Backend

- scaffold API service
- implement barcode, search, item detail, source detail, history, saved items
- add source provenance serialization
- add review endpoints and mapping override support

Data Workstream

- finalize MVP categories
- audit EPA datasets
- audit EPD sources and licensing
- define taxonomy and confidence rubric
- build first ingestion pipeline
- create source freshness policy

Design and Handoff

- complete Figma foundations
- complete wireframes and UI kit
- complete high-fidelity screens
- prototype exact-match and no-match flows
- deliver token names matching frontend theme keys

9. Current Implementation and Handoff Context

- The app runs in Expo and currently uses an in-app mock API by default.
- Implemented: onboarding screen styled to match screenshot-based mockup, permissions/capture screen styled to match screenshot-based mockup, tab screens Home/Search/Saved/Profile, second-level screens Barcode Scanner/Photo Capture/Candidate Results/Item Detail/Metric Detail/Source Detail/No Match, theme tokens with primary brand blue #004D77, local mock data and standalone mock server.

- Assets added: assets/opening-hero.png and assets/capture-hero.png.
- Behavior note: the onboarding title changed from Green Zone to Green Life. The clean logo asset has not yet been provided, so current logo marks are code-drawn approximations.
- Limitation: this session did not have a callable Figma MCP tool attached, even though the figma skill instructions were available.